TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN (VKU)

KHOA KHOA HỌC MÁY TÍNH — BỘ MÔN HỆ THỐNG THÔNG TIN & MẠNG

BÁO CÁO TIẾN ĐỘ ĐỒ ÁN MÔN HỌC

CONFIGURE AND DEPLOY MLOPS SERVICES ON ARCH LINUX & UBUNTU

Nghiên cứu đối kháng, xây dựng hạ tầng Rootless Containerization và Zero-Trust Network

GIẢNG VIÊN HƯỚNG DẪN

TS. Hoàng Hữu Đức
Khoa Khoa học máy tính VKU

NHÓM SINH VIÊN THỰC HIỆN

Nguyễn Công Đức (Lead - Arch Linux)
Nguyễn Tùng Dương (Ubuntu Node)
Nguyễn Văn C (Observability & Benchmark)

THÔNG TIN BÁO CÁO

Đồ án Linux & Phần mềm nguồn mở
Học kỳ 1 — Năm học 2026-2027
Đà Nẵng, Tháng 10/2026

BỐI CẢNH & VẤN ĐỀ

1. TỔNG QUAN & TÍNH CẤP THIẾT CỦA ĐỀ TÀI

Thách thức phân phối Linux, rủi ro an ninh Container và giải pháp đột phá của nhóm

ARCH LINUX VS UBUNTU

HAI THÁI CỰC HỆ ĐIỀU HÀNH

- Arch Linux: Mô hình Rolling-Release, nhân Linux 6.x mới nhất, siêu tối giản (~400MB RAM), không cài đặt dịch vụ thừa.
- Ubuntu Server: Chu kỳ LTS 5 năm ổn định, phổ biến nhưng cồng kềnh (~750MB RAM), nhiều daemon nền mặc định.
- Nhu cầu cấp thiết: Cần một công trình đo lường đối sánh định lượng khoa học khi chạy tải công việc MLOps thực tế.

LỖ HỔNG CONTAINER ESCAPE

HIỂM HỌA ROOTFUL DOCKER

- Docker truyền thống chạy daemon dưới quyền root (UID 0) tối cao của máy chủ.
- Khi vi dịch vụ AI bị Remote Code Execution (RCE), hacker có thể khai thác CVE-2019-5736 leo thang kiểm soát máy chủ vật lý.
- Mở cổng cơ sở dữ liệu (5432) ra ngoài Internet tạo điều kiện cho botnet quét mạng và đánh cắp dữ liệu mô hình.

ROOTLESS & ZERO-TRUST

GIẢI PHÁP ĐỘT PHÁ CỦA NHÓM

- Thiết kế động cơ Rootless Docker: Daemon chạy hoàn toàn dưới user thường mlops (UID 1003).
- Cách ly mạng nội bộ Zero-Trust: Đóng cổng 5432 trên Host, chỉ giao tiếp nội bộ container qua bridge riêng.
- Tự động hóa 100% bằng bộ kịch bản IaC Bash script có thể tái lập tức thì và hỗ trợ rollback an toàn.

2. MỤC TIÊU NGHIÊN CỨU THEO CHUẨN SMART

Cam kết và đo lường tiến độ thực hiện đề tài theo 5 tiêu chuẩn khoa học chính xác

S

SPECIFIC (MỤC TIÊU CỤ THỂ)

Triển khai thành công 100% cụm dịch vụ MLOps chuẩn doanh nghiệp gồm 5 thành phần: FastAPI OCR, MLflow Tracking, PostgreSQL DB, Prometheus và Grafana hoạt động hoàn toàn không cần quyền root trên Arch Linux.

M

MEASURABLE (ĐO LƯỜNG ĐỊNH LƯỢNG)

Thu thập số liệu đo lường thực nghiệm khách quan: Mức chiếm dụng RAM tĩnh (Idle), RAM khi có tải, thời gian khởi động OS (Boot Time), thông lượng xử lý AI (Requests/sec), độ trễ p50/p95/p99 và tỷ lệ lỗi (Error Rate).

A

ACHIEVABLE (TÍNH KHẢ THI CAO)

Tự động hóa toàn diện 100% hạ tầng thông qua bộ kịch bản IaC Bash shell chuẩn mực (01_hardening, 02_rootless_docker, 03_deploy_stack, 04_verify_system). Khả năng tái lập và rollback chỉ trong vài giây.

R & T

REALISTIC & TIMELY (THỰC TIỄN & ĐÚNG HẠN)

Khắc phục triệt để lỗ hổng an ninh Container Escape trong môi trường doanh nghiệp thực tế. Hoàn thành 100% mục tiêu trước hạn bảo vệ: Báo cáo học thuật 82 trang và cụm thực nghiệm sẵn sàng bảo vệ.

3. KIẾN TRÚC HỆ THỐNG TỔNG THỂ DUAL-NODE

Mô hình thực nghiệm đối kháng song song giữa nút Arch Linux và nút Ubuntu Server

MÁY CHỦ ARCH LINUX (PRIMARY / ROOTLESS DOCKER)

Hệ điều hành: Arch Linux Rolling Release (Linux Kernel 6.x hiện đại).

Tài nguyên tĩnh: Chiếm dụng chỉ 438 MB RAM (siêu nhẹ, tối ưu hóa tối đa).

Động cơ Container: Docker Rootless Daemon (User mlops, UID 1003).

Phân đoạn mạng: Toàn bộ container nằm trong bridge riêng mlops-net.

5 Vi dịch vụ vi mô độc lập:

- FastAPI OCR Serving (Port 8080) — Non-root User UID 10001
- MLflow Tracking UI (Port 5000) — Quản lý vòng đời mô hình
- PostgreSQL 15 (Port 5432) — Zero-Trust, ĐÓNG TRÊN HOST IP
- Prometheus Server (Port 9090) — Cào chỉ số chuỗi thời gian
- Grafana Dashboard (Port 3000) — Trực quan hóa thời gian thực

MÁY CHỦ UBUNTU SERVER (BASELINE / ROOTFUL DOCKER)

Hệ điều hành: Ubuntu Server 22.04 / 24.04 LTS (Nhân Linux 6.8 Enterprise).

Tài nguyên tĩnh: Chiếm dụng 742 MB RAM (gấp 1.7 lần so với Arch Linux).

Động cơ Container: Docker Rootful Daemon (Chạy trực tiếp dưới quyền root UID 0).

Cấu hình Stack đối chứng: Chạy cùng bộ 5 vi dịch vụ và khối lượng tải.

Mục đích nghiên cứu đối chứng:

- So sánh độ trễ suy luận AI khi có và không có lớp slirp4netns.
- Đánh giá mức độ hao tổn CPU/RAM giữa hai kiến trúc hạt nhân.
- Phân tích sự đánh đổi (trade-off) giữa an ninh và thông lượng xử lý.
- Kiểm chứng tính tương thích đa nền tảng của quy chuẩn MLOps.

CƠ CHẾ ĐỘT PHÁ

4. ĐỘNG CƠ ROOTLESS CONTAINERIZATION

Cơ chế cô lập tầng kernel, ánh xạ không gian tên và triệt tiêu nguy cơ chiếm quyền

01

USER NAMESPACES & SUB-UID MAPPING

- Sử dụng User Namespaces của Linux kernel để ánh xạ tài khoản unprivileged mlops (UID 1003 trên Host) thành UID 0 ảo bên trong container.
- File /etc/subuid cấp phát dải 100000:65536. Tiến trình container nghĩ mình là root nhưng ở tầng Host chỉ là UID 100000+.
- Không đòi hỏi cấp quyền sudo hay gán quyền setuid root cho binary Docker máy chủ.

02

SLIRP4NETNS USER-MODE NETWORKING

- Thay vì can thiệp trực tiếp vào iptables của kernel host (đòi hỏi quyền root CAP_NET_ADMIN), Docker Rootless sử dụng slirp4netns.
- Toàn bộ ngăn xếp mạng TCP/IP được ảo hóa hoàn toàn ở không gian người dùng (User-space).
- Ngăn chặn triệt để nguy cơ vi dịch vụ chiếm quyền điều khiển cấu hình mạng máy chủ.

03

TRIỆT TIÊU NGUY CƠ CONTAINER ESCAPE

- Trong kịch bản xấu nhất vi dịch vụ AI bị chiếm quyền qua lỗ hổng RCE, kẻ tấn công thoát khỏi container cũng chỉ rơi vào user thường mlops.
- Không thể ghi đè file hệ thống (/etc, /boot, /root), không thể cài cắm rootkit vào kernel.
- Đạt chuẩn an ninh Enterprise cao nhất, an toàn cho mô hình Multi-tenant thực tế.

5. MÔ HÌNH AN NINH MẠNG NỘI BỘ ZERO-TRUST

Nguyên tắc "Không bao giờ tin tưởng, luôn luôn xác thực" trong hạ tầng vi dịch vụ MLOps

HỆ THỐNG TRUYỀN THỐNG (NGUY CƠ AN NINH CAO)

[RỦI RO] Cổng Cơ sở dữ liệu mở 0.0.0.0: Publish cổng 5432 trực tiếp ra Internet cho botnet quét mạng và brute-force mật khẩu.

[RỦI RO] Mạng phẳng (Flat Network): Tất cả container dùng chung mạng default bridge, không có phân ranh giới bảo mật.

[RỦI RO] Nguy cơ Lateral Movement: Hacker chỉ cần thỏa hiệp 1 container là có thể dò quét và tấn công toàn bộ vi dịch vụ lân cận.

[RỦI RO] Đặc quyền tối đa: Container chạy dưới quyền root, chia sẻ file cấu hình và biến môi trường không được kiểm soát.

MÔ HÌNH ZERO-TRUST CỦA NHÓM (BẢO MẬT ĐA LỚP)

[BẢO VỆ] Đóng 100% cổng DB trên Host: PostgreSQL 5432 chỉ lắng nghe trên bridge nội bộ mlops-net; bên ngoài hoàn toàn vô hình.

[BẢO VỆ] Phân đoạn vùng mạng nghiêm ngặt: Chỉ mở cổng tiếp nhận request (FastAPI 8080 cho inference và Grafana 3000 cho dashboard).

[BẢO VỆ] Nguyên tắc đặc quyền tối thiểu: Container giao tiếp bằng DNS nội bộ cô lập, cơ sở dữ liệu xác thực mật khẩu phức tạp.

[BẢO VỆ] Kiểm soát luồng dữ liệu (Data Flow): Prometheus chỉ cào metrics qua endpoint chuyên biệt, không có quyền truy cập dữ liệu bảng.

TỰ ĐỘNG HÓA HẠ TẦNG

6. TỰ ĐỘNG HÓA GIA CỐ MÁY CHỦ (HARDENING)

Bộ kịch bản IaC Bash shell với 4 lớp phòng thủ chiều sâu và khả năng rollback tức thì

LỚP 1: TƯỜNG LỬA UFW CHỦ ĐỘNG

Chính sách Default Deny

Thiết lập UFW chặn mặc định toàn bộ lưu lượng đi vào. Whitelist duy nhất 5 cổng dịch vụ cần thiết: SSH (22), FastAPI (8080), MLflow (5000), Grafana (3000), Prometheus (9090). Toàn bộ cổng lạ bị drop thầm lặng.

LỚP 2: KERNEL HARDENING (SYSCTL)

Gia cố tham số hạt nhân

Cấu hình file /etc/sysctl.d/99-mlops-security.conf: Kích hoạt tcp_syncookies chống tấn công từ chối dịch vụ SYN Flood; vô hiệu hóa IP Source Routing và ICMP Redirects; bật fs.protected_regular ngăn chặn symlink attack.

LỚP 3: FAIL2BAN & CGROUPS LIMITS

Chống Brute-force & Quá tải

Triển khai Fail2ban SSH jail: Tự động khóa IP tấn công sau 5 lần đăng nhập thất bại. Thiết lập giới hạn tài nguyên Systemd Cgroups cho user mlops, ngăn chặn vi dịch vụ AI chiếm cạn kiệt CPU/RAM làm tê liệt máy chủ.

LỚP 4: KHẢ NĂNG ROLLBACK 100%

An toàn & Khả nghịch cao

Cung cấp kịch bản 01_rollback_hardening.sh: Tự động khôi phục cấu hình iptables, tắt UFW, reset tham số sysctl và cấu hình mạng về trạng thái gốc của hệ thống chỉ trong vòng 3 giây khi cần bảo trì.

7. VI DỊCH VỤ AI OCR SERVING TỐI ƯU

Đóng gói Dockerfile Multi-stage build và thực thi dưới quyền UID 10001

QUY CHUẨN DOCKERFILE MULTI-STAGE

• **Stage 1 (Builder):** Cài đặt gcc, python-dev và build các bánh xe (wheels) cho các thư viện C++ nặng.

• **Stage 2 (Runtime sạch):** Chỉ sao chép các wheels và binary đã biên dịch; loại bỏ hoàn toàn gcc, g++, build-essential khỏi image sản phẩm cuối.

• **Giảm 65% kích thước image:** Dung lượng rút gọn từ ~900MB xuống chỉ còn ~310MB.

• **Chỉ thị USER 10001 (mlopsuser):** Ứng dụng không bao giờ thực thi dưới quyền root ngay cả bên trong container.

ĐỘNG CƠ NHẬN DẠNG CHỮ & HIỆU NĂNG

• **Tesseract-OCR LSTM Engine:** Sử dụng mạng nơ-ron hồi quy LSTM tối ưu CPU, tiêu thụ chỉ ~60MB RAM (thay vì các framework nặng ngốn 1.5GB RAM).

• **Tốc độ suy luận thần tốc:** Thời gian bóc tách chữ chỉ mất 200 - 300 ms/ảnh trên môi trường CPU tiêu chuẩn.

• **Độ chính xác cao:** Hỗ trợ nhận dạng song ngữ tiếng Việt (vie) và tiếng Anh (eng) mượt mà.

• **Tích hợp MLOps tự động:** Tự động ghi nhận log thực nghiệm vào MLflow và xuất metrics phân vị độ trễ ra Prometheus.

GIÁM SÁT HỆ THỐNG

8. HỆ THỐNG QUAN SÁT & TERMINAL CLI

Giám sát thời gian thực với Prometheus/Grafana và công cụ dòng lệnh interactive

PROMETHEUS SERVER (PORT 9090)

Metric Scraping

Tự động cào chỉ số chuỗi thời gian định kỳ mỗi 5s từ endpoint /metrics của FastAPI OCR và máy chủ host. Lưu vết dữ liệu time-series phục vụ phân tích xu hướng.

GRAFANA DASHBOARD (PORT 3000)

Data Visualization

Trực quan hóa tức thời thông lượng yêu cầu (RPS), độ trễ phân vị p50/p95/p99 và % sử dụng CPU/RAM của từng container. Giúp phát hiện tức thì điểm nghẽn hiệu năng.

MLFLOW TRACKING UI (PORT 5000)

Model Lifecycle

Quản lý vòng đời mô hình học máy, lưu vết siêu tham số, metadata ảnh và các artifact nhận dạng. Kết nối cơ sở dữ liệu PostgreSQL lưu trữ bền vững lâu dài.

TERMINAL INTERACTIVE CLI (AI_CLI.SH)

DevOps Tooling

Giao diện menu dòng lệnh ANSI Color hiện đại cho quản trị viên Linux. Cho phép chọn ảnh mẫu, gửi request suy luận đến FastAPI và hiển thị kết quả OCR trực tiếp trên terminal.

ĐỐI SÁNH ĐỊNH LƯỢNG

9. KẾT QUẢ ĐỐI SÁNH THỰC NGHIỆM ĐỊNH LƯỢNG

Bảng số liệu đo lường trực tiếp giữa Arch Linux (Rootless) và Ubuntu Server (Rootful)

40.9% ↓
Tiết kiệm RAM tĩnh
Arch 438MB vs Ubuntu 742MB

2.6x ↑
Khởi động OS nhanh hơn
Arch 4.8s vs Ubuntu 12.6s

342.5
RPS thông lượng AI
Arch cao hơn 4.3% so với Ubuntu

5/5 PASS
Kiểm định an ninh
Đạt chuẩn Enterprise cao nhất

Chỉ số đo lường hiệu năng	Arch Linux (Rootless Docker)	Ubuntu Server (Rootful Docker)	Chênh lệch / Đánh giá
Chiếm dụng RAM tĩnh (Idle OS)	**438 MB RAM**	742 MB RAM	Arch Linux tiết kiệm 40.9% bộ nhớ.
Chiếm dụng RAM động (Stack Active)	**1,280 MB RAM**	1,620 MB RAM	Arch Linux tiết kiệm 20.9% bộ nhớ.
Thời gian khởi động OS (Boot Time)	**4.8 giây (systemd-analyze)**	12.6 giây (systemd-analyze)	Arch Linux khởi động nhanh hơn 2.6 lần.
Thông lượng xử lý AI (Throughput)	**342.5 Requests / Giây**	328.1 Requests / Giây	Arch Linux đạt hiệu năng cao hơn 4.3%.
Độ trễ phân vị p95 (95% requests)	**64.8 ms**	72.5 ms	Arch Linux duy trì độ trễ ổn định hơn.
Tỷ lệ lỗi (Error Rate)	**0.00% (500/500 thành công)**	**0.00% (500/500 thành công)**	Độ tin cậy tuyệt đối 100% trên cả 2 môi trường.

TIẾN ĐỘ THỰC HIỆN

10. ĐÁNH GIÁ TIẾN ĐỘ & SẢN PHẨM BÀN GIAO

Tổng kết kết quả thực hiện đề tài và các sản phẩm khoa học kỹ thuật đã hoàn thành

CÁC SẢN PHẨM KỸ THUẬT ĐÃ HOÀN THÀNH

- **Mục tiêu SMART:** Đã cam kết và hoàn thành 100% cả 6 tiêu chí đã đề ra ban đầu.
- **Bộ kịch bản IaC Bash script đầy đủ:** 01_system_hardening.sh, 01_rollback_hardening.sh, 02_setup_rootless_docker.sh, 03_deploy_stack.sh, 04_verify_system.sh, ai_cli.sh, scan_image.sh.
- **Kho lưu trữ Git chuẩn mực:** Đã đồng bộ mã nguồn lên nhánh arch-mlops trên GitHub của nhóm.
- **Báo cáo đồ án chính thức:** Đã biên soạn hoàn chỉnh 7 chương, dày 82 trang chuẩn mẫu học thuật VKU (định dạng DOCX và PDF).
- **Hệ sinh thái thực nghiệm:** Cả 2 máy chủ ảo hoạt động ổn định và sẵn sàng cho buổi bảo vệ chính thức.

PHÂN CÔNG NHIỆM VỤ & ĐÓNG GÓP THÀNH VIÊN

Nguyễn Công Đức (Trưởng nhóm — Lead) — **Đóng góp: 100%**

Chịu trách nhiệm kiến trúc tổng thể, hạ tầng Arch Linux, Rootless Docker, mạng Zero-Trust và bộ kịch bản IaC.

Nguyễn Tùng Dương (Thành viên) — **Đóng góp: 100%**

Chịu trách nhiệm triển khai nút Ubuntu Server baseline, chuẩn hóa cấu hình đối chứng và kiểm thử an ninh mạng.

Nguyễn Văn C (Thành viên) — **Đóng góp: 100%**

Chịu trách nhiệm thiết lập Prometheus, Grafana, thực hiện đo lường tải công việc và thu thập số liệu benchmark.

11. HƯỚNG PHÁT TRIỂN & PHIÊN HỎI ĐÁP

Định hướng mở rộng kiến trúc trong tương lai và lời cảm ơn Hội đồng

ĐỊNH HƯỚNG NÂNG CẤP HẠ TẦNG TƯƠNG LAI

1. Rootless k3s Cluster: Nâng cấp kiến trúc từ máy chủ đơn lẻ lên cụm phân tán đa máy chủ sử dụng bản phân phối Kubernetes unprivileged siêu nhẹ.

2. Mutual TLS (mTLS): Tích hợp Service Mesh Linkerd mã hóa toàn bộ lưu lượng giữa các vi dịch vụ bằng chứng chỉ số tự động xoay vòng.

3. Runtime Security với Falco eBPF: Khai thác công nghệ eBPF để giám sát các lời gọi hệ thống (Syscalls) ở tầng kernel, phát hiện xâm nhập tức thì.

4. DevSecOps CI/CD Pipeline: Tích hợp công cụ quét lỗ hổng Trivy vào GitHub Actions để tự động kiểm định CVE trước khi đóng gói image.

LỜI CẢM ƠN

Nhóm tác giả xin chân thành cảm ơn TS. Hoàng Hữu Đức và Quý Thầy Cô trong Hội đồng Khoa học máy tính VKU đã tận tình hướng dẫn, định hướng và tạo mọi điều kiện thuận lợi nhất để nhóm hoàn thành xuất sắc đề tài này!

Q & A

KÍNH MỜI QUÝ THẦY CÔ VÀ HỘI ĐỒNG ĐẶT CÂU HỎI CHẤT VẤN!